DevOps Fundamentals Assignment 3

Healthcare Application Modernization

A DevOps operating model for faster, safe and auditable patient-facing services

Field	Details
Student name	Mohit
Institution	National Institute of Information Technology
Course / Subject	Java Full Stack
Instructor	Mr. Pravin
Submission date	October, 04, 2026

Table of Contents

Table of Contents 2
1. Executive Summary 3
2. Business Scenario and Objectives 3
2.1 Scenario 3
2.2 Objectives 3
3. Stakeholders 3
4. Current Challenges and Risks 4
4.1 Challenges 4
4.2 Risks of the Transformation Itself 4
5. Assumptions and Constraints 5
5.1 Given Requirements (from the assignment) 5
5.2 Assumptions (made by the author) 5
5.3 Constraints 6
6. Proposed Solution (Step-by-Step Approach) 6
6.1 Design Principles and Risk-Based Change 6
6.2 DevOps Operating Model 6
6.3 Secure and Compliant Delivery Pipeline 8
6.4 Reliability and Safe Release Practices 9
6.5 Key Performance Indicators (KPIs) 9
6.6 Traceability Requirements 10
6.7 Continuous Improvement Framework 12
7. Implementation Plan 13
7.1 Milestones (illustrative) 14
8. Measurable Success Criteria 14
9. Recommendations and Lessons Learned 14
9.1 Recommendations 15
9.2 Lessons Learned 15
9.3 Limitations of This Plan 15

1. Executive Summary

Problem. A healthcare organisation wants to deliver patient-facing services (such as a patient portal, appointment booking and telehealth) faster, without weakening security, regulatory compliance, reliability or the ability to prove what changed and who approved it.

Solution. A DevOps operating model with four pillars: (1) cross-functional product teams supported by a shared platform team and specialist advisers, (2) a **risk-based** change process in which low-risk changes flow automatically and high-risk changes get deeper review, (3) a delivery pipeline that produces security checks and audit evidence automatically, and (4) a continuous improvement cycle driven by balanced KPIs. Traceability requirements link every production change back to its request, code, review, tests and deployment.

Expected result. More frequent and safer releases, stronger security, audit evidence available in hours instead of weeks, and a steady habit of improvement. Section 8 defines how success will be measured.

2. Business Scenario and Objectives

2.1 Scenario

The organisation wants faster delivery of patient-facing services while keeping security, compliance, reliability and auditability. In healthcare these goals seem to conflict: speed looks risky, and controls look slow. The assignment asks for a DevOps operating model, KPIs, traceability requirements and a high-level continuous improvement framework.

Simple explanation of an operating model: it is the blueprint for how people, processes, technology and rules work together to deliver software. A DevOps operating model describes who owns what, how work flows to production, and how it is controlled and improved.

2.2 Objectives

#	Objective	Why it matters
O1	Deliver patient-facing improvements faster	Patients and clinicians benefit sooner
O2	Keep patient data secure and private	A breach harms patients and breaks the law
O3	Meet regulatory and policy requirements	Non-compliance brings fines and loss of trust
O4	Keep services reliable	Patients depend on access, sometimes urgently
O5	Make every change auditable and traceable	Auditors and regulators must be able to see who changed what and why
O6	Build a repeatable way to keep improving	A one-off project fades; a habit lasts

3. Stakeholders

A stakeholder is anyone affected by, or responsible for, how services are delivered.

Stakeholder	Role	What they need
Patients	Use patient-facing services	Easy, reliable, secure access; privacy protected
Clinicians and care staff	Rely on the applications in daily care	Dependable systems; changes that do not disrupt care
Executive leadership	Sponsor and fund the change	Business value with controlled risk
Product owners	Decide features and priorities	Faster delivery of valuable features

Stakeholder	Role	What they need
Development teams	Build and change the software	Clear rules, fast feedback, modern tools
QA / Testers	Verify quality	Test automation and safe test data
Operations / SRE	Run services in production	Stable releases, monitoring, runbooks
Security team	Protect systems and data	Security checks built into delivery
Privacy and Compliance officers	Ensure legal and policy compliance	Evidence and controls they can trust
Clinical safety officer	Judge patient-safety impact of changes	Early involvement in high-risk changes
Internal audit and regulators	Check that rules are followed	Complete, retrievable evidence
Third-party vendors and integration partners	Provide systems such as health records and payment	Stable interfaces; advance notice of changes

4. Current Challenges and Risks

The assignment does not describe the current process in detail. The challenges below are common in healthcare organisations and are treated as assumptions about the current state (Assumption A1).

4.1 Challenges

#	Challenge	Impact	Risk
C1	Slow, infrequent releases of patient-facing services	Patients wait for improvements; features arrive in large risky batches	High
C2	Speed is seen as the enemy of safety and compliance	Teams avoid change; delivery slows further	High
C3	Compliance evidence collected by hand, after the fact	Slow approvals; gaps found during audits	High
C4	Security testing late or inconsistent	Vulnerabilities reach production, putting patient data at risk	High
C5	Real patient data used in test environments	Privacy breach risk; compliance failure	High
C6	Weak traceability between requests, code, approvals and releases	Cannot prove what changed, who approved it or why	High
C7	Silos between development, operations, security and compliance	Hand-offs, delays and blame	Medium
C8	Legacy systems and integrations with health record systems	Changes ripple unexpectedly; slow testing	High
C9	Uptime expectations for care services	Fear of downtime makes teams cautious	Medium
C10	One approval process for every change, whatever its risk	Small safe changes wait as long as dangerous ones	Medium
C11	Skills gap in automation and security practices	Slow adoption; dependence on a few experts	Medium

Risk level is a judgement based on the likely effect on patients, data and regulation. It is not measured data.

4.2 Risks of the Transformation Itself

#	Risk	Level	Mitigation
T1	Faster change harms patients or breaks compliance	High	Risk-based change classes; automated gates; clinical safety review for high-risk changes; pilot first
T2	Staff or auditors distrust automation	High	Involve compliance and audit early; run an audit dry run; show evidence quality
T3	Legacy and vendor systems cannot change quickly	High	Start with patient-facing channels; wrap legacy systems with tested interfaces
T4	KPIs encourage the wrong behaviour (speed at any cost)	Medium	Pair speed KPIs with stability and safety KPIs; review for gaming
T5	Tool-first approach without culture change	High	Operating model and ownership first; tools support them
T6	Cost and skills	Medium	Small standard toolset; training; platform team to share effort
T7	Evidence gets collected but nobody reviews it	Medium	Named owners; regular review forums; audit dry runs

5. Assumptions and Constraints

This section separates what the assignment **requires** from what this report **assumes**, so the two are never confused.

5.1 Given Requirements (from the assignment)

#	Requirement
R1	A healthcare organisation wants faster delivery of patient-facing services
R2	Security, compliance, reliability and auditability must be maintained
R3	Propose a DevOps operating model
R4	Identify KPIs
R5	Define traceability requirements
R6	Create a high-level continuous improvement framework
R7	Deliver a report, workflow diagrams, recommendations, lessons learned and an implementation plan

5.2 Assumptions (made by the author)

#	Assumption	Effect if wrong
A1	The assignment gives no details of the current state; challenges, baselines and numbers are **illustrative**	Real values come from measurement in the first phase
A2	The organisation's country, and so the applicable laws, are not stated. Examples include HIPAA (USA), GDPR (EU) and India's Digital Personal Data Protection Act. This report uses general compliance requirements; the exact laws must be checked	Specific controls, retention periods and reporting duties would change
A3	Patient-facing services means a patient portal, mobile app, appointment booking and telehealth, not clinical decision-making software	Software that acts as a medical device would need stricter, specialised controls
A4	The organisation is mid to large size, with several product teams and a central IT function	Team structure would scale down or up
A5	Services connect to a health record system and other vendor systems through interfaces	More or fewer integration constraints
A6	Cloud or hybrid hosting allows automated environments	On-premises only would need more setup time

#	Assumption	Effect if wrong
A7	Privacy, security, compliance and clinical safety roles already exist	They would have to be created or outsourced
A8	Tools named (for example GitLab, Terraform, Prometheus, Grafana) are examples only	Any equivalent tools can be used
A9	The timeline of 18 months is an estimate	Real durations depend on size, skills and legacy

5.3 Constraints

- **Patient safety comes first:** no speed target justifies a change that could harm patients.
- **Privacy and data protection:** patient data is highly sensitive and tightly regulated.
- **Regulation and audit:** evidence must be complete, tamper-resistant and kept for the required period.
- **Availability:** care depends on services being available, often around the clock.
- **Separation of duties:** the person who writes a change cannot be the only one to approve it.
- **Legacy and vendors:** older systems and third parties limit the pace of change.
- **Budget, skills and time:** limited resources.
- **Scope:** this is a high-level design exercise; no real systems are changed.

6. Proposed Solution (Step-by-Step Approach)

The solution has seven parts. The assignment's four required outputs are the operating model (6.2), KPIs (6.5), traceability requirements (6.6) and the continuous improvement framework (6.7). Sections 6.1, 6.3 and 6.4 provide the security, compliance and reliability foundations they rely on.

6.1 Design Principles and Risk-Based Change

Key idea: speed and control are not opposites if control is **automatic** and **proportionate to risk**. A tiny text change on a booking page should not wait as long as a change to how medical data is stored.

Class	Examples (illustrative)	Approval	Evidence
Low risk	Text, styling and small fixes with no data or security impact	Automatic, if all pipeline gates pass	Pipeline records stored automatically
Normal risk	New feature on a patient-facing screen; performance change	Peer review plus service owner approval	Pipeline records plus approval record
High risk	Changes to patient data handling, login and access, integrations, or anything affecting clinical information	Security, privacy and clinical safety review plus service owner	Pipeline records, review notes, test evidence, rollback plan
Emergency	Urgent fix for a live problem or security issue	Fast-track approval by an on-call authority	Same evidence, completed after the fix at the latest

Principles: patient safety first; privacy by design; automate evidence instead of collecting it by hand; small changes are safer than big ones; everyone shares responsibility for quality, security and compliance.

6.2 DevOps Operating Model

The model has four layers: **People** (team structure), **Process** (how work flows), **Technology** (the shared platform) and **Governance** (how risk is controlled).

Component	Description	Purpose
Stream-aligned product teams	Cross-functional teams (developers, testers, operations skills, product owner) each owning one patient-facing service end to end	Removes hand-offs; one team is accountable for results
Platform team	A team that builds and runs the shared "paved road": pipeline, environments as code, monitoring, logging, secrets management	Product teams get safe defaults without building tools themselves
Enabling specialists	Security, privacy, compliance and clinical safety advisers who coach teams and set policy	Brings expertise early without becoming a bottleneck
Risk-based governance	Automated policies and risk classes replace one-size-fits-all committee approval; a light forum handles high-risk changes	Control without delay for low-risk work
Service ownership	Each team owns its service in production, with on-call responsibility, supported by the platform team	Builders feel the results of their work
Shared metrics	All teams use the same KPIs (Section 6.5)	One version of the truth; shared goals

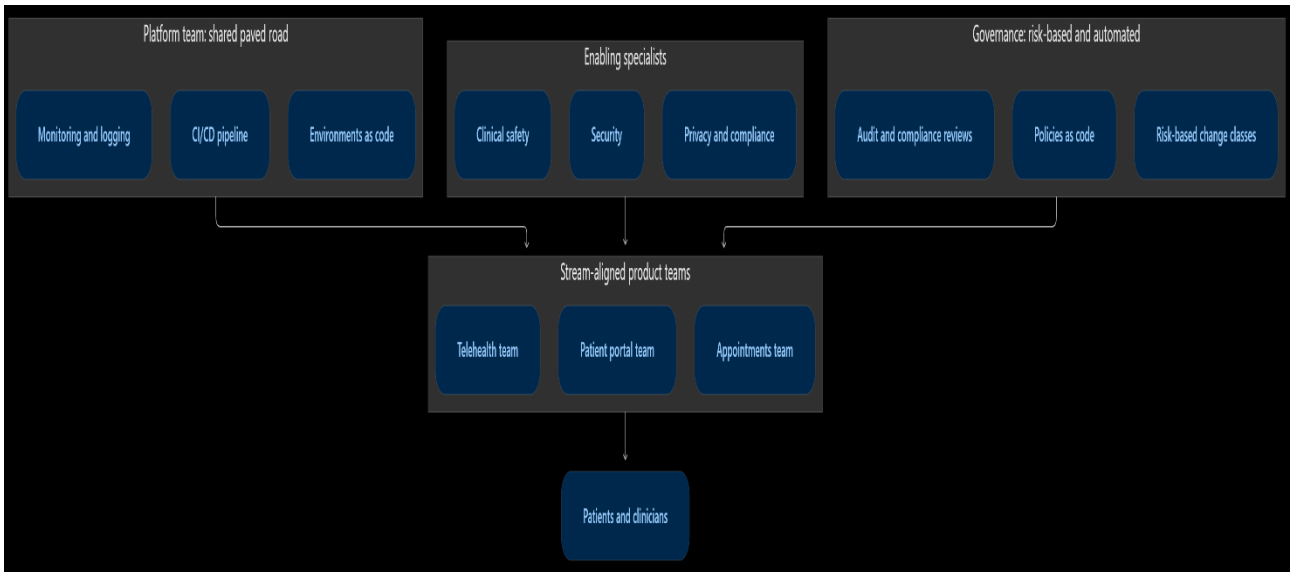


DIAGRAM 1: DEVOPS OPERATING MODEL

How work flows from idea to production

1. A work item with a ticket ID is approved and prioritised.
2. A developer works on a branch and opens a merge request.
3. A reviewer other than the author reviews and approves it.
4. The pipeline builds, runs tests, and runs security and privacy checks.
5. The change is deployed to test and staging environments using synthetic (not real) patient data.
6. The change's risk class decides the approval path (Section 6.1).
7. The change is released gradually to production and monitored.
8. If health checks fail, it is rolled back automatically or by a quick decision; otherwise the trace record is stored and feedback is collected.

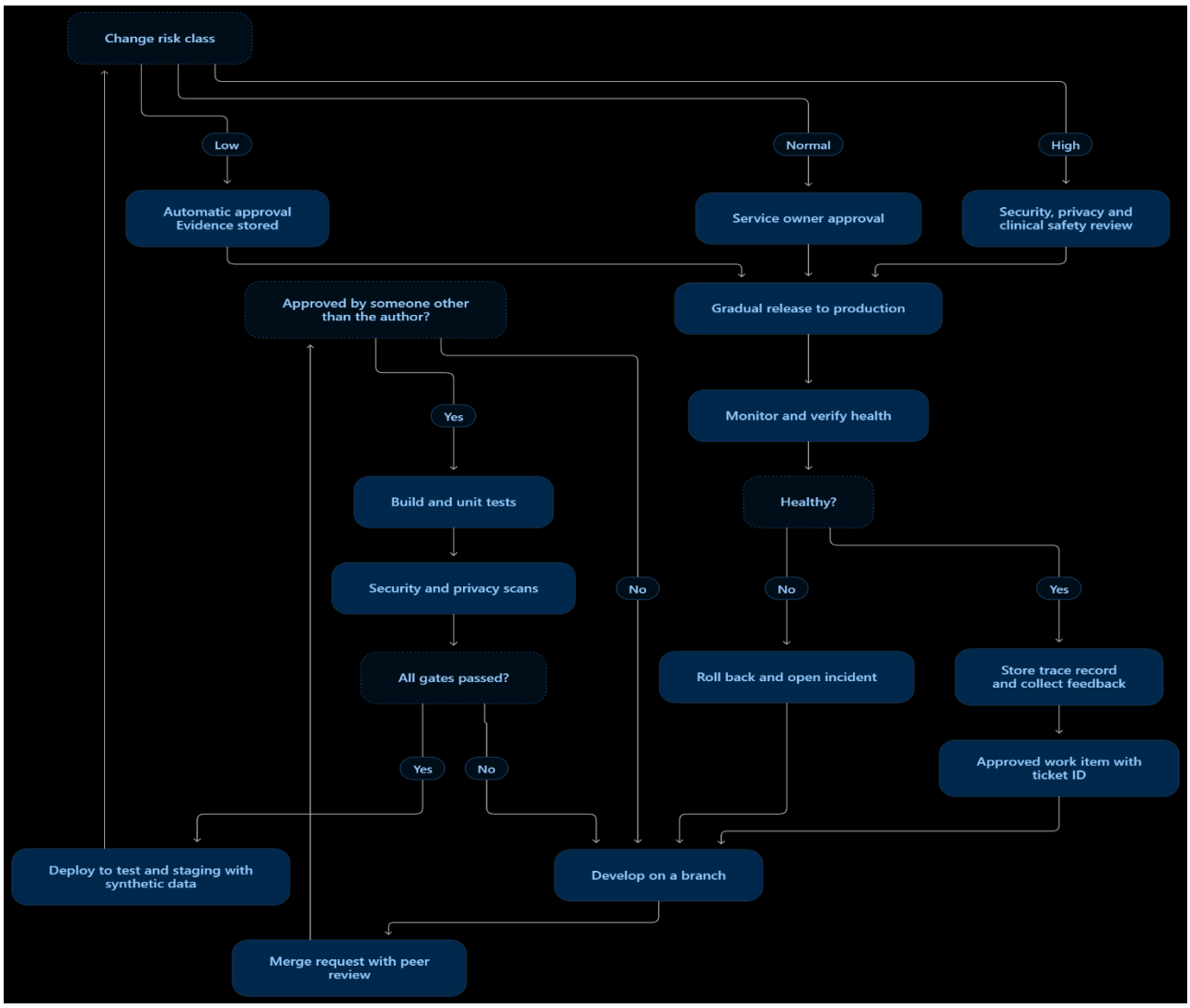


DIAGRAM 2: DELIVERY WORKFLOW WITH CONTROLS

Roles and responsibilities

Role	Responsibility
Product owner	Prioritises work; accepts delivered value
Product team	Builds, tests, releases and runs the service; owns quality
Platform team	Provides the pipeline, environments, monitoring and logging; keeps them reliable
Security	Sets security policy and gates; advises on threats; reviews high-risk changes
Privacy and Compliance	Defines compliance requirements; checks evidence; supports audits
Clinical safety officer	Reviews changes that could affect patient safety
Service owner	Approves normal-risk changes; accountable for the service
Internal audit	Independently checks that the process works as described

6.3 Secure and Compliant Delivery Pipeline

Simple explanation: compliance as code means turning rules into automatic checks, so the pipeline refuses non-compliant changes and records the proof. Compliance becomes a by-product of delivery, not a separate burden.

Control	How it is automated	Evidence produced
Code review and separation of duties	Merge requests need approval from someone other than the author; direct pushes to main are blocked	Approval record with names and times
Static code analysis (SAST)	Runs on every change	Scan report stored per build
Dependency and container scanning	Checks libraries and images for known vulnerabilities	Vulnerability report; list of components
Secret detection	Blocks passwords and keys in code	Scan log
Dynamic testing (DAST)	Tests the running test environment for common attacks	Test report
Privacy checks	Rules check that no real patient data enters non-production; logs are checked for personal data	Check results and data-masking records
Automated testing	Unit, integration and regression tests	Test results stored per build
Infrastructure as code	Environments created from reviewed, versioned files	Change history of the environment
Access control	Least-privilege access; production access limited and logged; emergency ("break-glass") access recorded and reviewed	Access logs and review records
Encryption and secrets management	Data encrypted in transit and at rest; secrets in a vault	Configuration records

- **Test data:** use synthetic or properly masked data in all non-production environments.
- **Vulnerability handling:** agreed deadlines for fixing critical issues (for example within 7 days, illustrative).
- **Threat modelling:** a short review for new features that handle patient data.
- **Immutable artifacts:** build once, deploy the same package everywhere, so what was tested is what is released.

6.4 Reliability and Safe Release Practices

Practice	What it means	Benefit
Service level objectives (SLOs)	Agreed targets such as 99.9% availability for the patient portal	Shared definition of "reliable enough"
Monitoring and alerting	Metrics, logs and traces; alerts on patient impact, such as failed logins or bookings	Problems seen before patients report them
Canary and blue-green releases	Release to a few users first, or switch between two environments	Problems affect few patients; no downtime
Feature flags	Turn features on or off without redeploying	Instant kill switch; separates deployment from release
Automated rollback	Return to the previous version when health checks fail	Fast recovery
Incident management	Clear roles, severity levels, communication and blameless postmortems	Consistent, fast response and learning
Backup and disaster recovery	Regular backups; tested recovery plans	Data and service can be restored
Release timing	Avoid risky changes at peak clinical hours; plan around known busy periods	Fewer disruptions to care

6.5 Key Performance Indicators (KPIs)

Simple explanation: a KPI is a number that shows whether the operating model is working. The set below is **balanced**, so improving speed is never measured alone.

Area	KPI	Why it matters	Source
Speed	Deployment frequency	How often value reaches patients	Pipeline records
Speed	Lead time for changes (commit to production)	How quickly ideas become live	Ticket and pipeline timestamps
Stability	Change failure rate	Share of releases that cause problems	Incident and rollback records
Stability	Time to restore service (MTTR)	How fast we recover	Incident log
Reliability	Availability of patient-facing services	Whether patients can use the service	Monitoring and SLO reports
Security	Time to fix critical vulnerabilities	Window of exposure	Vulnerability tracker
Security	Pipelines with mandatory security scans	Coverage of the controls	Pipeline configuration reports
Compliance and audit	Changes with a complete trace record	Auditability	Trace and audit checks
Compliance and audit	Evidence produced automatically	Less manual effort and fewer gaps	Evidence inventory
Compliance and audit	Time to produce audit evidence on request	Audit readiness	Audit exercises
Compliance and audit	Audit findings on change management	Independent view of control quality	Audit reports
Quality	Regression tests automated; defects found in production	Confidence in releases	Test tool and defect tracker
Patient outcome	Patient satisfaction and task completion (for example booking completed)	Whether faster delivery helps patients	Surveys and analytics
People	Team satisfaction and on-call load	Sustainability of the model	Staff survey and rota

Avoiding the wrong incentives

When a number becomes a target, people may game it. To prevent this, speed KPIs are always read together with a stability or safety KPI, and KPIs are used to improve processes, not to rank people.

Speed metric	Paired with	Why
Deployment frequency	Change failure rate	Releasing often is only good if releases succeed
Lead time for changes	Defects found in production	Faster is not better if quality drops
Evidence produced automatically	Audit findings	Automation is useful only if auditors trust the evidence

6.6 Traceability Requirements

Simple explanation: traceability means being able to follow a change in both directions: from any production change back to who asked for it, who wrote it, who approved it, and what tests it passed; and from any requirement forward to where it is running. In healthcare, it lets the organisation answer auditors, regulators and incident investigators with facts.

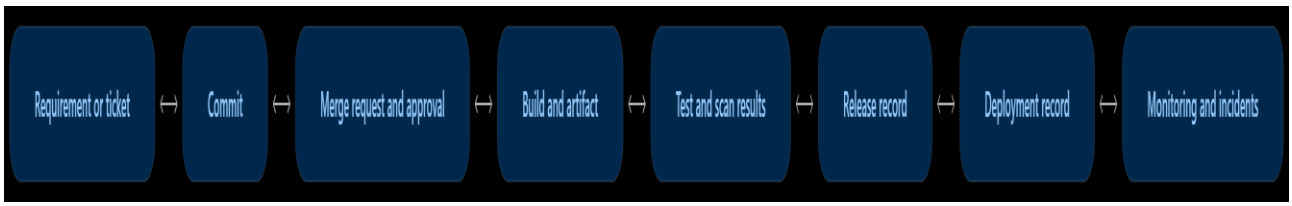


DIAGRAM 3: TRACEABILITY CHAIN

ID	Requirement	How it is met	Record kept
TR1	Every change is linked to an approved work item	Ticket ID is mandatory in commits and merge requests; checked automatically	Ticket and commit history
TR2	Every change is reviewed and approved by someone other than the author	Protected branches; mandatory approvals	Merge request approval record
TR3	Every build is versioned and linked to its source	Immutable artifact with version, commit ID and checksum	Artifact registry metadata
TR4	Test, security and privacy results are stored for every build	Pipeline stores reports with the build	Retained reports
TR5	Every deployment is recorded: what, where, when, by whom, approved by whom	Pipeline logs deployments automatically	Deployment log
TR6	Infrastructure and configuration changes are versioned	Infrastructure as code in version control	Version history
TR7	Access to production systems and patient data is logged and reviewed	Central access logs; regular reviews	Access logs and review sign-offs
TR8	Audit logs are tamper-evident, time-synchronised and kept for the required period	Protected, append-only log storage	Log retention records (period set by the applicable regulations)
TR9	Release notes and change records are generated, not typed	Created from tickets and commits	Release notes
TR10	Incidents are linked to the changes that may have caused them, and postmortem actions are tracked	Incident tool linked to deployment log	Incident and action records
TR11	Requirements that affect patient safety or privacy are linked to the tests that prove them	Requirement-to-test links in the test tool	Traceability matrix
TR12	Evidence can be produced quickly on request	Evidence stored in one searchable place	Audit request log (target: within 1 working day)

Example audit trail for one change (illustrative)

Step	Record
Requirement	HC-1042: allow patients to reschedule appointments online
Code change	Commit a1b2c3d: "feat(booking): add reschedule option (HC-1042)"
Review	Merge request !87, approved by a reviewer who is not the author
Build	Build #4521, artifact booking-2.8.0 with checksum
Tests and scans	Unit and regression tests passed; security scan: no critical findings; privacy check passed
Risk class and approval	Normal risk; approved by the service owner
Deployment	Released to production on a recorded date by the pipeline, gradually (canary)
After release	Health checks passed; no related incidents; dashboard link stored

```
# Example commit message with traceability
feat(booking): add reschedule option

Lets patients move an appointment online.
Refs: HC-1042

# Finding every change for a requirement (Git examples)
git log --oneline --grep="HC-1042"
git tag -a v2.8.0 -m "Release 2.8.0"            # mark the released version
git log v2.7.0..v2.8.0 --format="%h %an %s"     # what changed in a release
```

6.7 Continuous Improvement Framework

Simple explanation: continuous improvement means regularly asking "how are we doing and what is the next small thing to fix?" and acting on the answer. This framework follows the **Plan, Do, Check, Act** cycle at three levels.

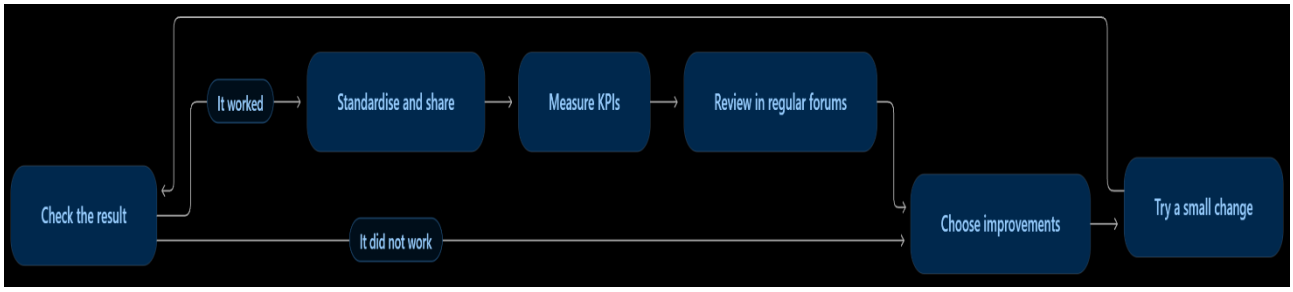


DIAGRAM 4: CONTINUOUS IMPROVEMENT CYCLE

Cadence	Forum	Inputs	Outputs
Every sprint (1 to 2 weeks)	Team retrospective	Team KPIs, issues, feedback	1 to 2 small improvement actions
After each incident	Blameless postmortem	Timeline, trace records, monitoring	Fixes and process improvements with owners
Monthly	Service review	KPIs, SLOs, incidents, patient feedback	Service improvement priorities
Quarterly	Operating model review	All KPIs, audit findings, maturity assessment	Changes to the operating model; investment decisions
Yearly	Independent audit and strategy review	Audit reports, regulatory changes	Compliance updates; roadmap for next year

How improvements are chosen and tracked

1. **Measure:** collect the KPIs from the pipeline and monitoring.
2. **Review:** discuss them in the regular forums above.
3. **Choose:** pick a small number of improvements; each has an owner and a due date in a shared improvement backlog.
4. **Try:** run a small change or experiment.
5. **Check:** compare KPIs before and after.
6. **Standardise or drop:** keep what works, document it, and share it with other teams.

Sources of improvement ideas

- KPIs and trends, value stream mapping (finding where changes wait), and incident postmortems.
- Patient and clinician feedback, support tickets and satisfaction surveys.

- Audit findings, security reviews and changes in regulation.

Simple maturity model

Level	Name	Description
1	Initial	Manual, inconsistent processes; evidence collected by hand; releases rare and risky
2	Managed	Version control and basic pipeline in use; some automated tests; traceability partial
3	Defined	Standard pipeline with security gates; risk-based approvals; automatic evidence; shared KPIs
4	Optimised	Fast, frequent, safe releases; continuous improvement habit; audit evidence on demand

The maturity of each team is assessed every quarter and used to decide where to coach and invest, not to rank teams.

7. Implementation Plan

The plan runs for about **18 months** (an assumption). It begins with a pilot service so the organisation learns safely.

Why pilot a non-clinical service first? A patient-facing but non-clinical service, such as appointment booking, carries lower patient-safety risk. It lets the organisation prove the model, fix mistakes cheaply, and win the trust of compliance and audit teams before moving to sensitive services.

Phase	Months	Activities	Output
1. Foundation	1 to 3	Assess current delivery and compliance process; measure baseline KPIs; agree risk classes with Security, Compliance and Clinical Safety; define traceability requirements; choose the pilot service; form the pilot team and a small platform team; initial training	Baselines; agreed risk classes; pilot team
2. Build the paved road	4 to 6	Build the pipeline with security and privacy gates; infrastructure as code; synthetic test data; audit logging; monitoring and SLOs; automatic evidence collection	Working pipeline for the pilot
3. Pilot delivery	7 to 12	Pilot releases regularly through the pipeline; risk-based approvals; first improvement cycles; audit dry run with Compliance and Internal Audit; onboard 2 to 3 more services	Proven model; audit dry-run report
4. Scale and optimise	13 to 18	Extend to remaining patient-facing services; quarterly operating model reviews; maturity assessments; prepare for external audit; share good practice	Most patient-facing services on the model; improvement habit

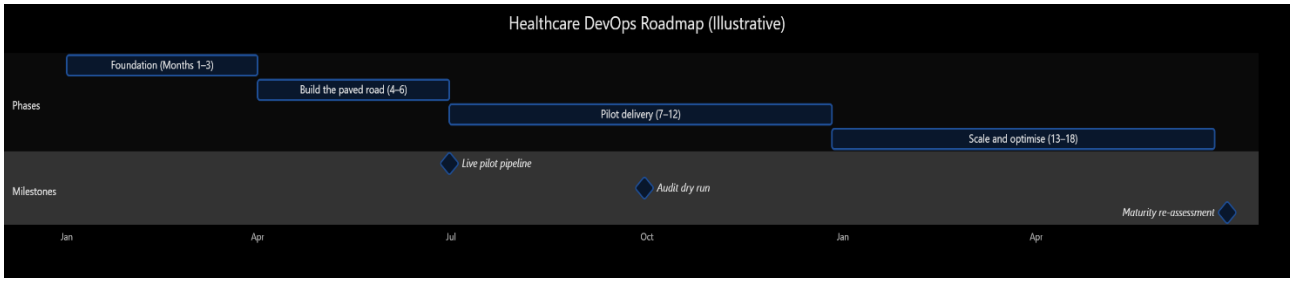


DIAGRAM 5: IMPLEMENTATION ROADMAP

7.1 Milestones (illustrative)

Milestone	Target state
Month 3	Baseline KPIs measured; risk classes and traceability requirements agreed
Month 6	Pilot pipeline with security and privacy gates and automatic evidence is live
Month 9	Pilot releasing at least monthly; internal audit dry run completed
Month 12	Pilot releasing fortnightly; 2 to 3 more services on the pipeline
Month 18	Most patient-facing services on the operating model; maturity level 3 or higher

8. Measurable Success Criteria

The assignment gives no baseline figures, so baselines are assumed (Assumption A1) and targets are illustrative. They should be re-based on real measurements in Phase 1.

#	Metric	Baseline (assumed)	Target	How measured
S1	Deployment frequency	Quarterly	Monthly by month 12; fortnightly by month 18	Pipeline records
S2	Lead time for normal changes	2 to 3 months	Under 2 weeks	Ticket and pipeline timestamps
S3	Change failure rate	About 20%	Under 10%	Incident and rollback records
S4	Time to restore a patient-facing service	1 to 2 days	Under 2 hours	Incident log
S5	Availability of patient-facing services	99.5%	99.9% or better	Monitoring
S6	Changes with a complete trace record	About 60%	100%	Audit sampling
S7	Audit evidence produced automatically	Mostly manual	At least 90%	Evidence inventory
S8	Time to produce audit evidence on request	Weeks	Under 1 working day	Audit exercises
S9	Critical vulnerabilities fixed within the agreed deadline	Not tracked	100%	Vulnerability tracker
S10	Pipelines with mandatory security scans	Partial	100%	Pipeline configuration reports
S11	Real patient data in non-production environments	Present in some	None	Data audits
S12	Separation-of-duties violations	Unknown	0	Approval and access logs
S13	Major audit findings on change management	Unknown	0	Audit reports
S14	Regression tests automated	Under 30%	At least 70%	Test tool
S15	Patient satisfaction with digital services	Baseline survey in Phase 1	Measurable improvement	Surveys
S16	Improvement actions completed on time	Not tracked	At least 80%	Improvement backlog

9. Recommendations and Lessons Learned

9.1 Recommendations

1. Classify changes by risk, so low-risk changes flow automatically and high-risk changes get deeper review.
2. Make compliance automatic: turn rules into pipeline checks that also produce the evidence.
3. Never use real patient data in test environments; use synthetic or masked data.
4. Pilot on a non-clinical, patient-facing service before moving to sensitive services.
5. Involve security, privacy, compliance and clinical safety from day one, and run an audit dry run before external audits.
6. Build a platform team so product teams get a safe, shared pipeline instead of each building their own.
7. Use balanced KPIs and pair speed metrics with stability and safety metrics.
8. Make improvement a routine, with owners, due dates and regular reviews.

9.2 Lessons Learned

These come from designing this plan, not from a live transformation.

- **Speed and compliance are not opposites.** Automatic controls and evidence can make both better.
- **One process for every change slows everything.** Risk-based classes give speed where it is safe and care where it is needed.
- **Evidence is a by-product, not a project.** If the pipeline records it, nobody has to chase it later.
- **What gets measured gets gamed.** Balanced, paired KPIs guard against bad incentives.
- **Patient safety is a design constraint,** not a checkbox at the end.
- **Trust takes time.** Showing auditors real evidence early builds the confidence the model needs.

9.3 Limitations of This Plan

- The applicable regulations are not stated; the plan uses general requirements that must be checked against the real laws and standards.
- Baselines, targets and the timeline are assumptions and need real measurement.
- Software that behaves as a medical device needs additional regulatory controls that are not designed here.
- Integration with health record systems (for example through health data standards) is not covered in detail.
- Costs and staffing levels are not estimated.
- Tool names are examples; a real selection would compare options against the organisation's needs.